

Training Dataset Annotation

Training Dataset Annotation

1. Course Content
2. Example Dataset
3. Annotating Traffic Sign Dataset
 - 3.1 Label Studio Introduction
 - 3.2 Start Label Studio
 - 3.3 Access Label Studio
 - 3.4 Create Project
 - 3.5 Import Images
 - 3.6 Label Settings
 - 3.7 Dataset Annotation
 - 3.8 Export Dataset
 - 3.9 Organize Dataset Directory Structure
 - 3.10 Dataset Configuration File
4. Annotating Lane Detection Dataset
 - 4.1 Create Project
 - 4.2 Organize Export Dataset
 - 4.3 Dataset Configuration File

[!IMPORTANT]

- The factory image already contains preset datasets required for training traffic sign recognition models and lane recognition models. This section tutorial is provided for users who need to train YOLOv11 models themselves and want to learn how to collect and annotate their own datasets.
- If you want to experience the complete functionality directly, you can skip this section!!!

1. Course Content

1. Master the data annotation method for datasets required to train YOLOv11 object detection models.

2. Example Dataset

- The factory image already contains datasets required for training YOLOv11 traffic sign recognition models and lane recognition models. You can find the corresponding datasets in the following paths on the vehicle or in the files of this section.
- Traffic sign dataset path

```
~/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset.zip
```

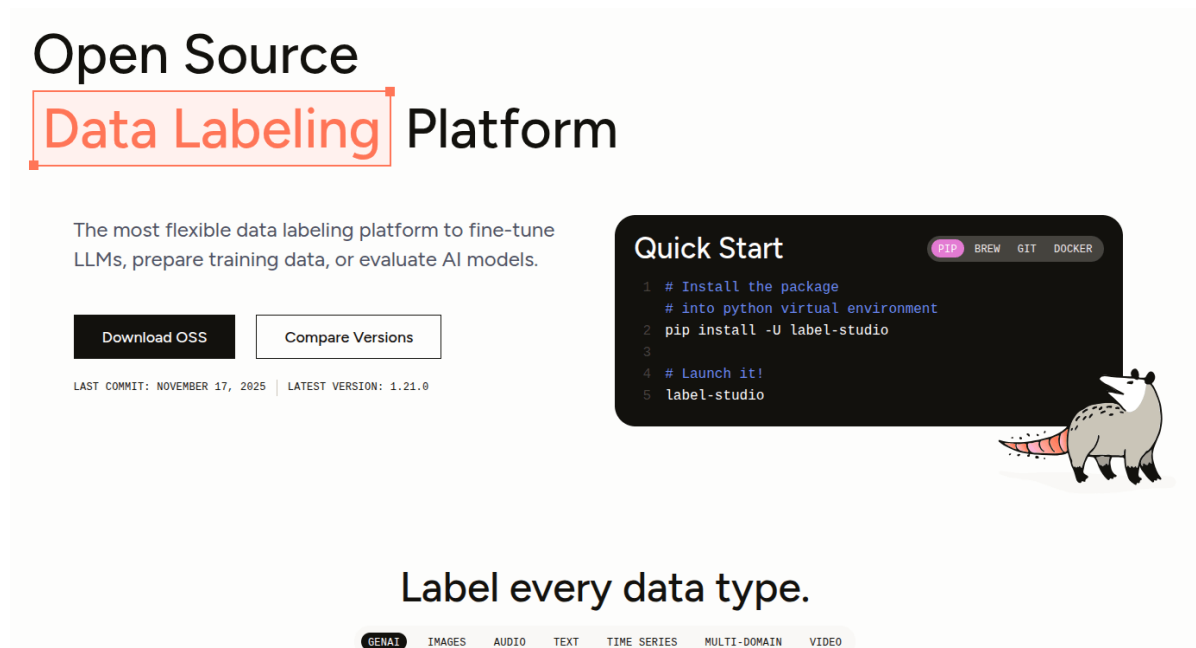
- Lane dataset path

```
~/yahboomM3_ws/yolo_train/train_lane/lane_dataset.zip
```

3. Annotating Traffic Sign Dataset

3.1 Label Studio Introduction

Label Studio is an open-source data annotation platform for data annotation and annotation task management, supporting various types of data input and annotation formats.



The screenshot shows the Label Studio website. At the top, it says "Open Source Data Labeling Platform" with "Data Labeling" in a red box. Below this, it states: "The most flexible data labeling platform to fine-tune LLMs, prepare training data, or evaluate AI models." There are two buttons: "Download OSS" and "Compare Versions". Below the buttons, it says "LAST COMMIT: NOVEMBER 17, 2025 | LATEST VERSION: 1.21.0". On the right, there is a "Quick Start" section with a terminal window showing the installation command:

```
1 # Install the package
2 # into python virtual environment
3 pip install -U label-studio
4 # Launch it!
5 label-studio
```

 There is also a cartoon dog illustration. At the bottom, it says "Label every data type." with a row of buttons: GENAI, IMAGES, AUDIO, TEXT, TIME SERIES, MULTI-DOMAIN, VIDEO.

[!NOTE]

The factory image environment has already installed Label Studio!!! If users need to set up a data annotation environment on their own computers, they need to install Label Studio first.

Installation command is as follows:

```
pip install label-studio
```

3.2 Start Label Studio

ROSMaster-M3 uses uv to manage Python package environments. Label Studio is installed in the uv environment of the yahboomM3 workspace.

- Activate uv environment

```
source /home/jetson/yahboomM3_ws/.venv/bin/activate
```

- Start Label Studio

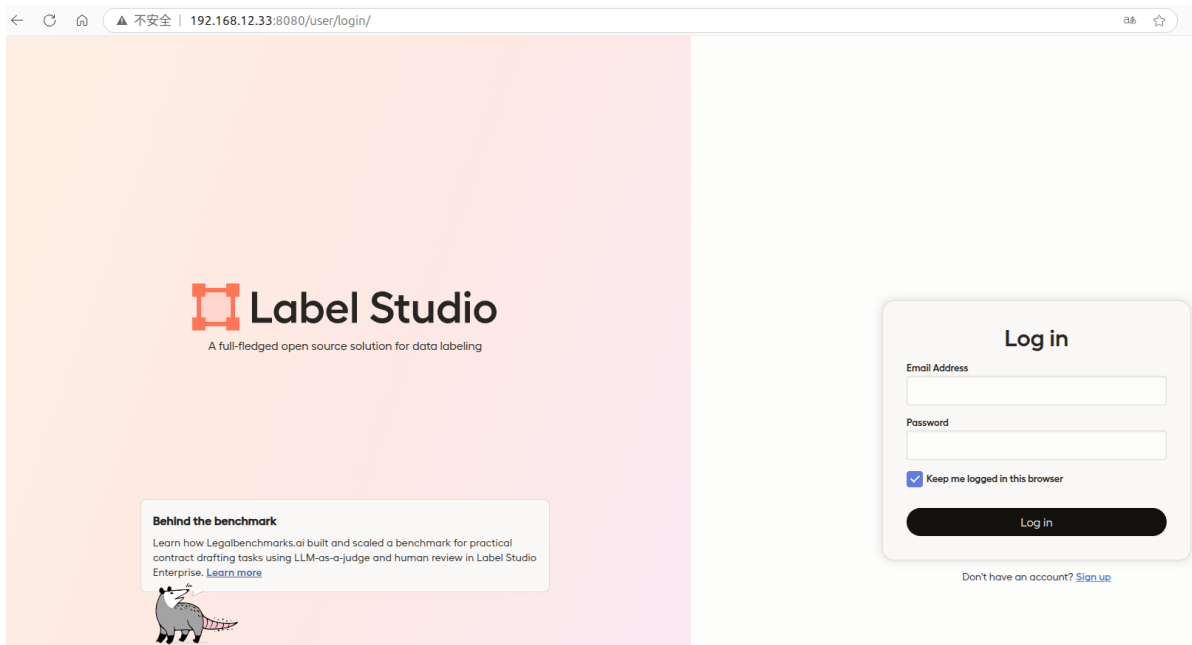
```
label-studio
```

```
jetson@yahboom: ~  
jetson@yahboom: ~ 112x24  
jetson@yahboom:~$ source /home/jetson/yahboomM3_ws/.venv/bin/activate  
(yahboomM3_ws) jetson@yahboom:~$ label-studio  
=> Database and media directory: /home/jetson/.local/share/label-studio  
=> Static URL is set to: /static/  
=> Database and media directory: /home/jetson/.local/share/label-studio  
=> Static URL is set to: /static/  
Read environment variables from: /home/jetson/.local/share/label-studio/.env  
get 'SECRET_KEY' casted as '<class 'str'>' with default ''  
Not in REPL -> leaving logger event level as is.  
Looking for locale 'en_US' in provider 'faker.providers.address'.  
Provider 'faker.providers.address' has been localized to 'en_US'.  
Looking for locale 'en_US' in provider 'faker.providers.automotive'.  
Provider 'faker.providers.automotive' has been localized to 'en_US'.  
Looking for locale 'en_US' in provider 'faker.providers.bank'.  
Specified locale 'en_US' is not available for provider 'faker.providers.bank'. Locale reset to 'en_GB' for this provider.  
Looking for locale 'en_US' in provider 'faker.providers.barcode'.  
Provider 'faker.providers.barcode' has been localized to 'en_US'.  
Looking for locale 'en_US' in provider 'faker.providers.color'.  
Provider 'faker.providers.color' has been localized to 'en_US'.  
Looking for locale 'en_US' in provider 'faker.providers.company'.  
Provider 'faker.providers.company' has been localized to 'en_US'.  
Looking for locale 'en_US' in provider 'faker.providers.credit_card'.  
Provider 'faker.providers.credit_card' has been localized to 'en_US'.
```

3.3 Access Label Studio

Ensure the computer and vehicle are on the same local area network. Enter the vehicle IP + port 8080 in the browser address bar. The vehicle IP can be viewed from the OLED screen in front of the vehicle or by opening a new terminal. Here we use `IP:192.168.12.33` as an example:

192.168.12.33:8080



- Factory preset account information (all locally stored information, no data leakage risk):

Account: yahboom@163.com
Password: yahboom@163.com

- If you need to create a new account, click `sign up`. If using the factory preset account, log in directly.



Label Studio

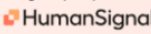
A full-fledged open source solution for data labeling

Did you know?

Label Studio now has a Starter Cloud offering optimized for small teams and projects. [Learn more](#)



Brought to you by



Log in

Email Address

Password

☒ Keep me logged in this browser

Log in

Don't have an account? [Sign up](#)



Label Studio

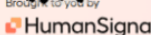
A full-fledged open source solution for data labeling

Did you know?

Label Studio now has a Starter Cloud offering optimized for small teams and projects. [Learn more](#)



Brought to you by



Sign Up

Email Address

Password

How did you hear about Label Studio?

Github

☐ Get the latest news from Heidi 

Create Account

Already have an account? [Log in](#)

After registration is completed, the website will automatically log in:

Label Studio



Heidi doesn't see any projects here!

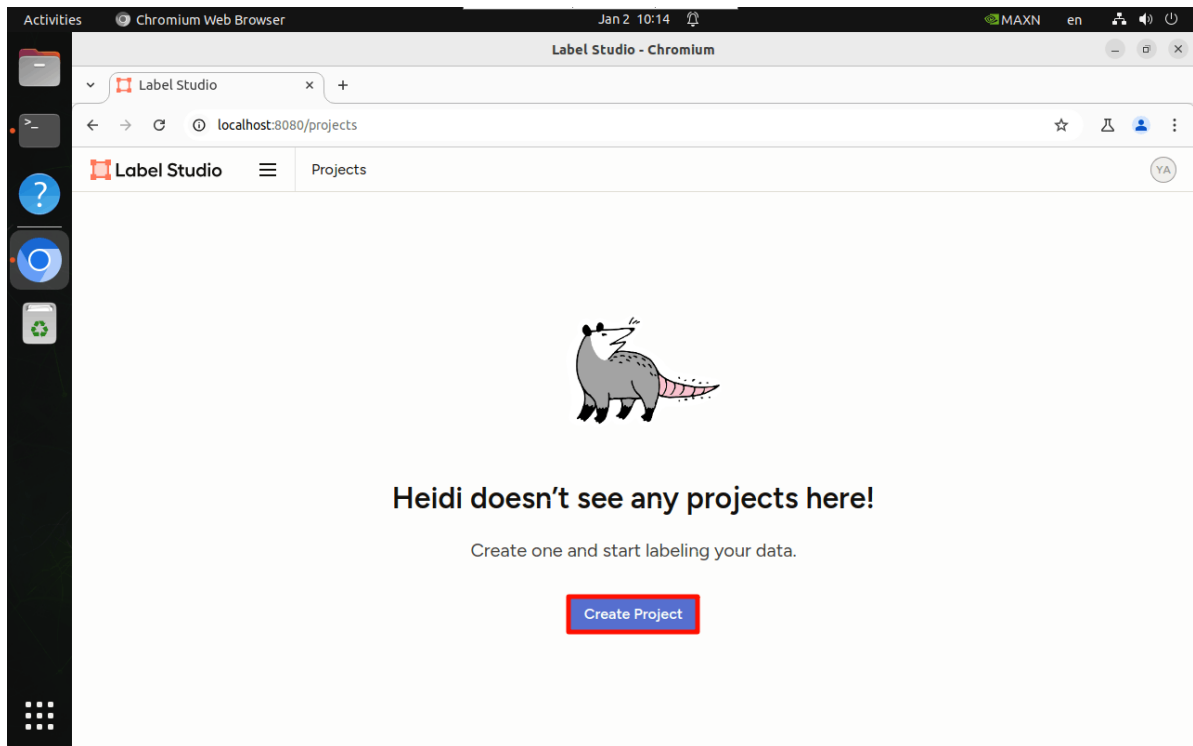
Create one and start labeling your data.

Create Project

Account & Settings

Log Out

3.4 Create Project

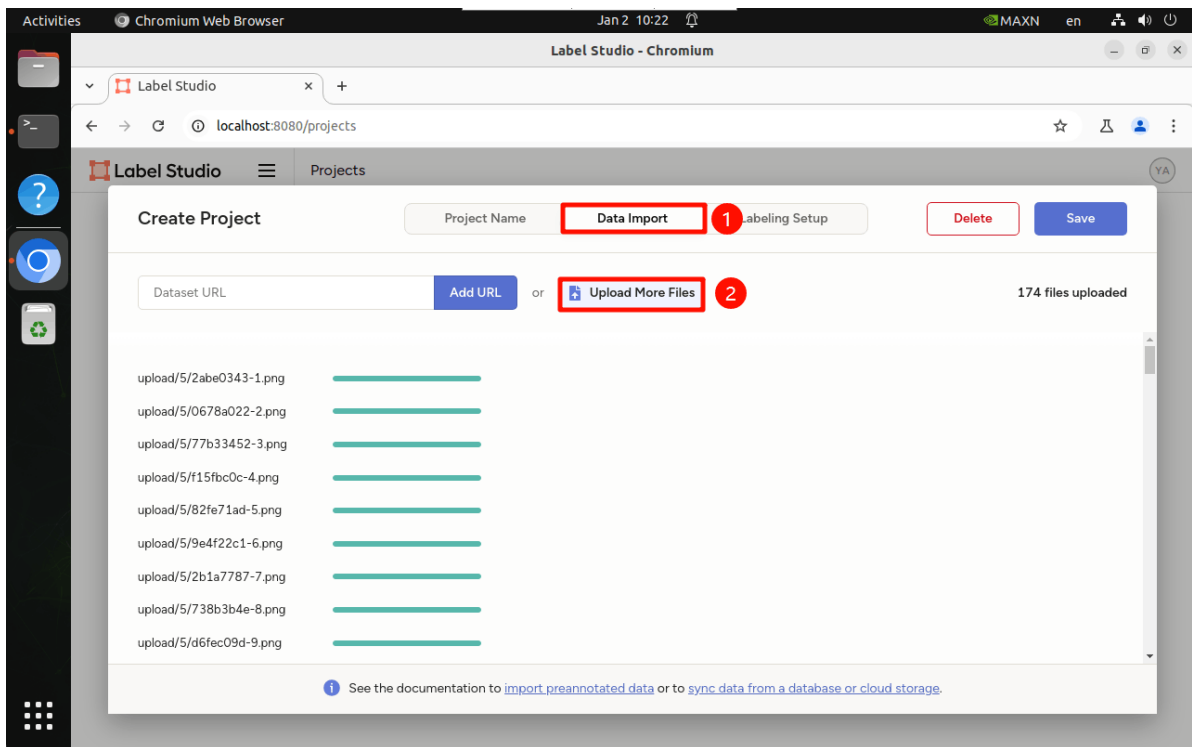


Project name can be named arbitrarily, name it according to your training set:

A screenshot of the 'Create Project' form in the Label Studio application. The form has three tabs: 'Project Name', 'Data Import', and 'Labeling Setup'. The 'Project Name' tab is active, and the 'Project Name' input field is highlighted with a red box and contains the text 'train_sign'. Below this is a 'Description' field with the placeholder text 'Optional description of your project'. Further down is a 'Workspace' dropdown menu with a red 'Enterprise' tag and the text 'Select an option'. At the bottom, there is a 'Did you know?' section with a small cartoon dinosaur character and a link to 'Learn more'.

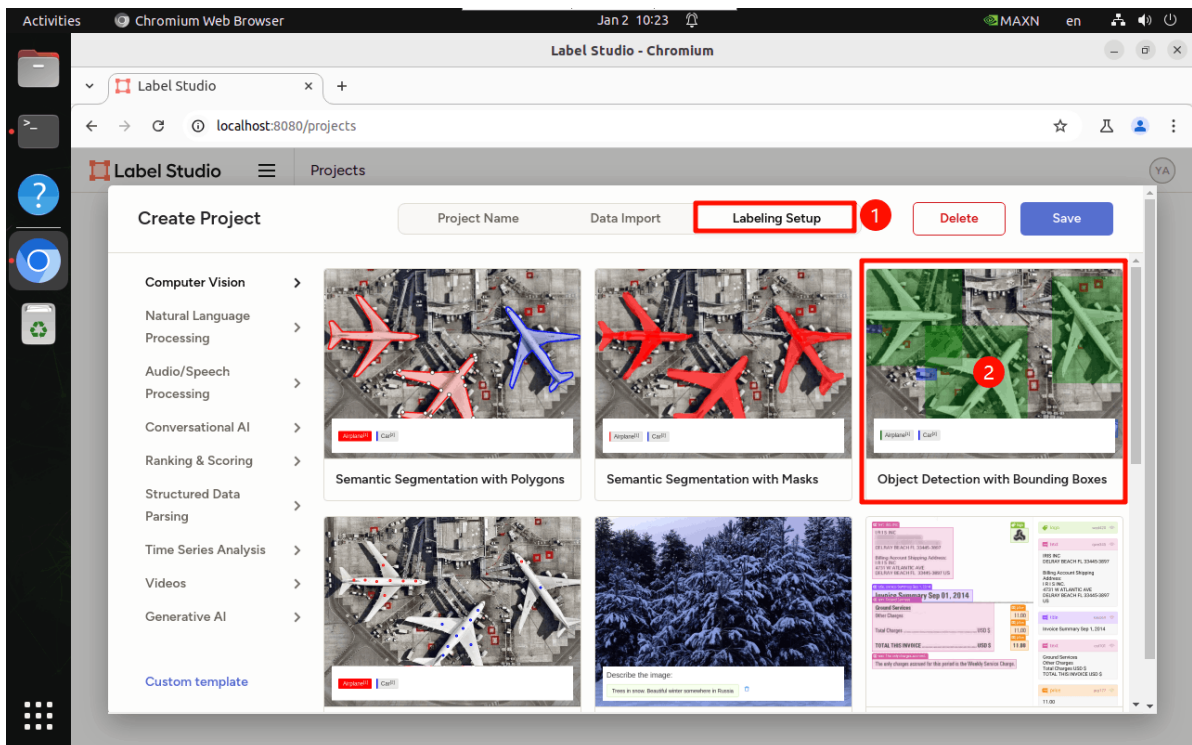
3.5 Import Images

- Import the images previously split from the video: Do not import too many images at once. It is recommended to select 50-100 images each time, then import multiple times.

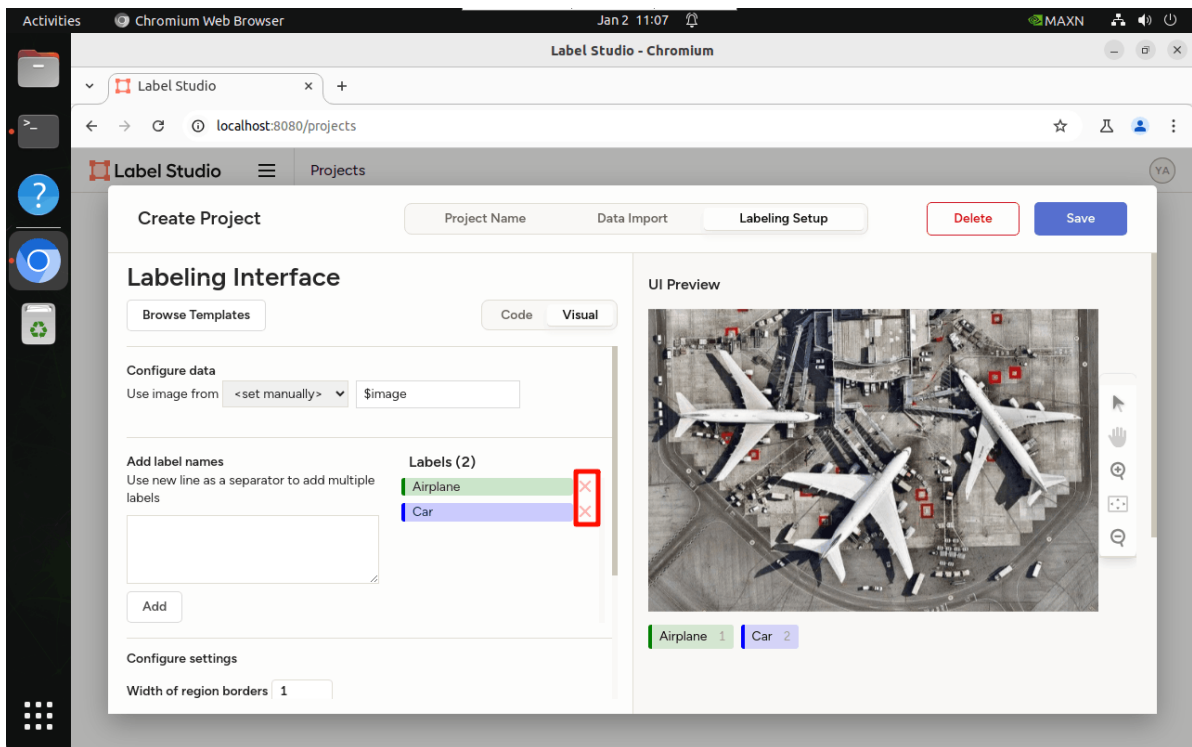


3.6 Label Settings

- Select "Object Detection" model



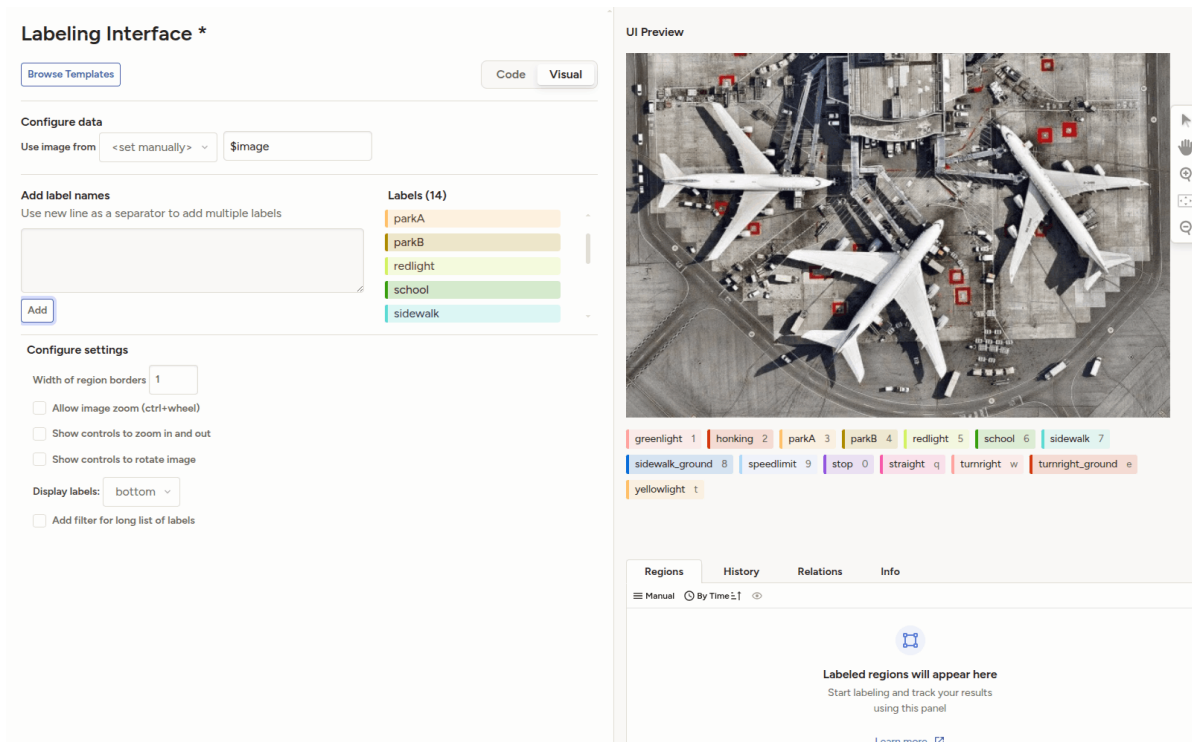
- Delete built-in labels:



- Enter a label name in **Add Label names**, then click **Add** to add it.
- In this example, we name all traffic signs and ground signs in the sandbox 配套地图.

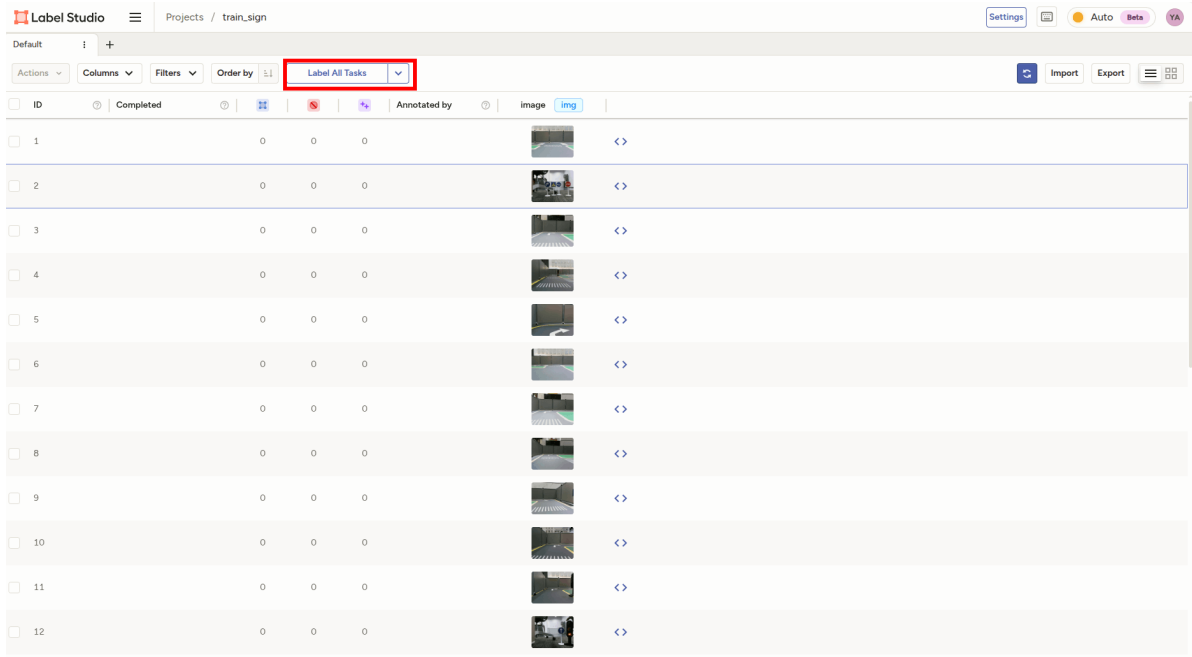
[!TIP]

- The label names here are not fixed, you can name them according to your needs.
- If users want to train other types of object detection models, and the detection targets are not traffic signs or lanes, name the labels according to the targets that need to be detected in your dataset.

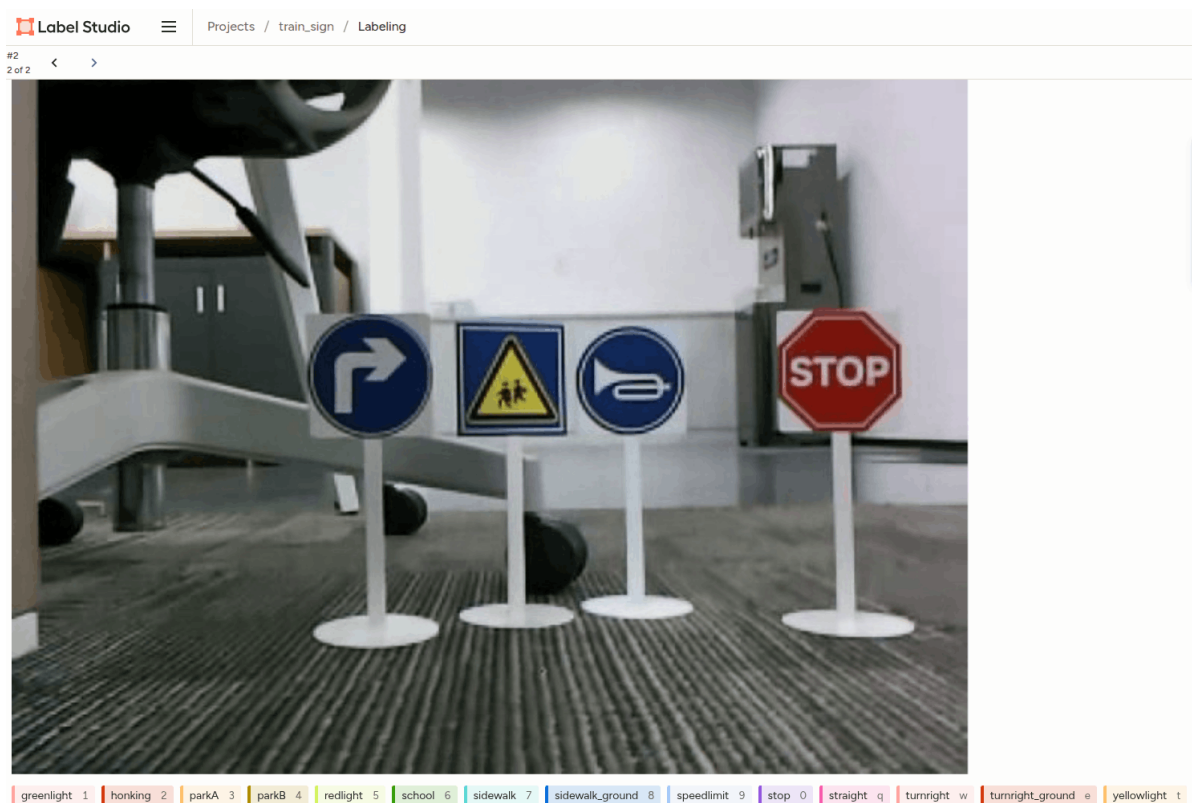


3.7 Dataset Annotation

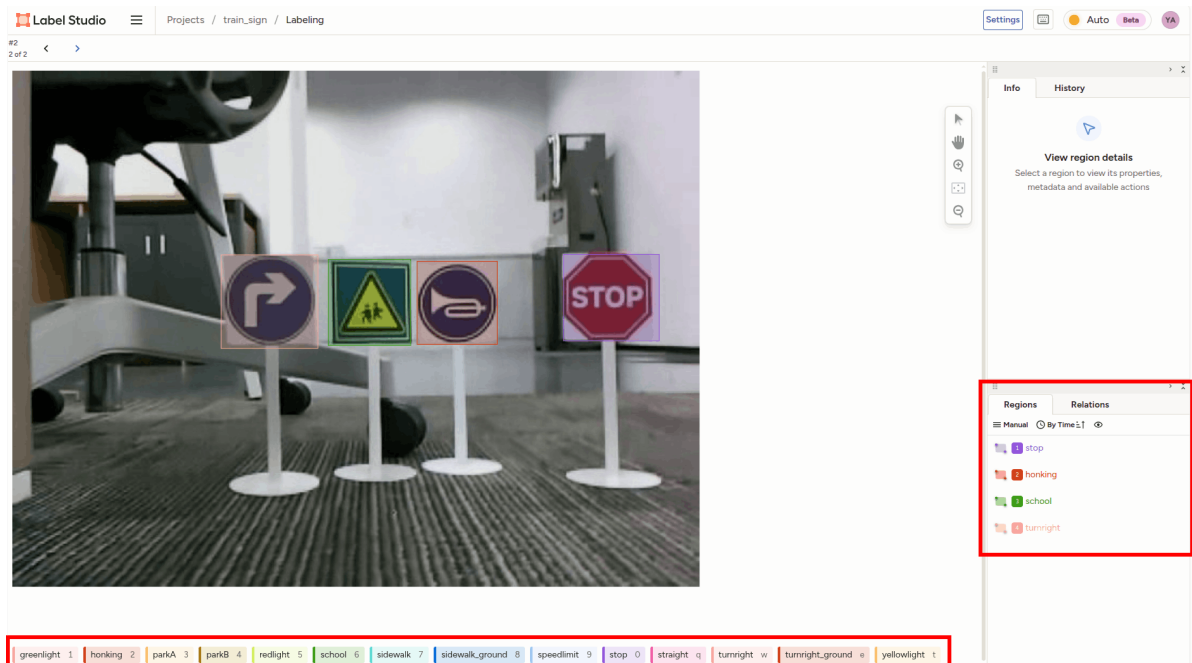
- Return to the main page, then click Annotate All Tasks:



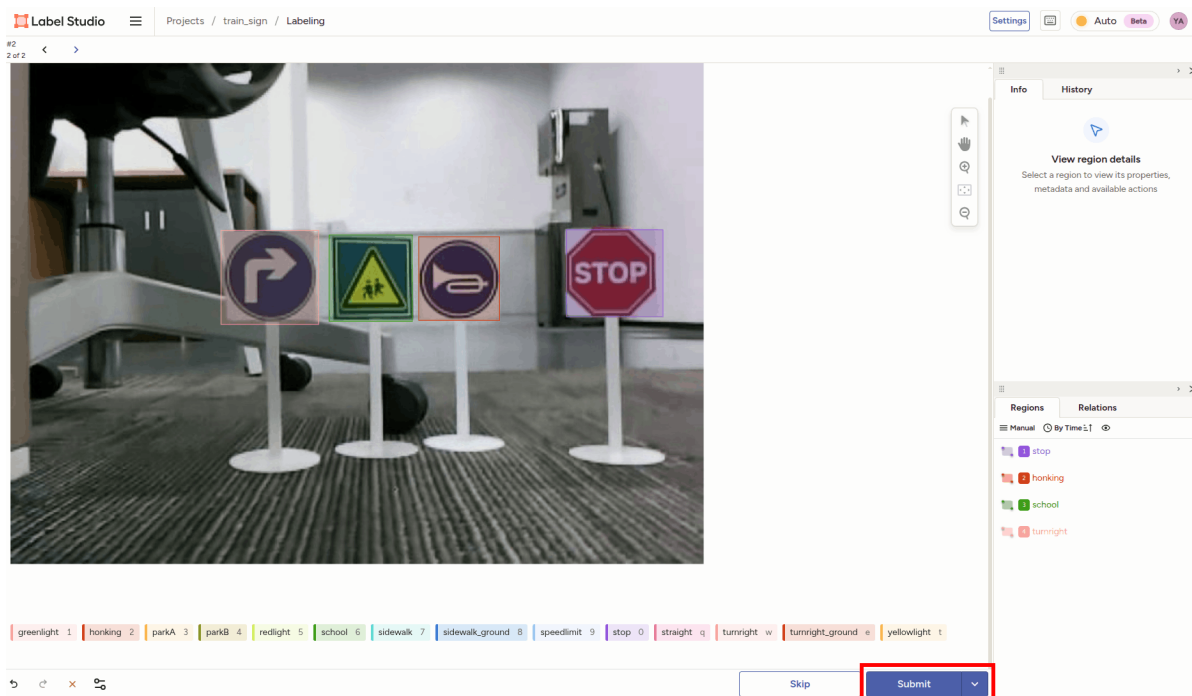
- The shortcut keys for the corresponding target categories are displayed at the bottom of the label. Press the corresponding shortcut key on the keyboard to quickly select the label.



- Then drag the mouse to select the target corresponding to the selection. There are 4 types of targets in the figure here. After marking is completed, the marked labels will be displayed in the **Regions** column on the right, where you can check whether the marking is correct.



- After marking is completed, click **Submit** to submit, and it will automatically jump to the next image. Repeat the annotation until all image data annotation is completed.



3.8 Export Dataset

- Select the project to export, click **Export** in the upper right corner

Default

+

Actions

Columns

Filters

Order by

Label All Tasks

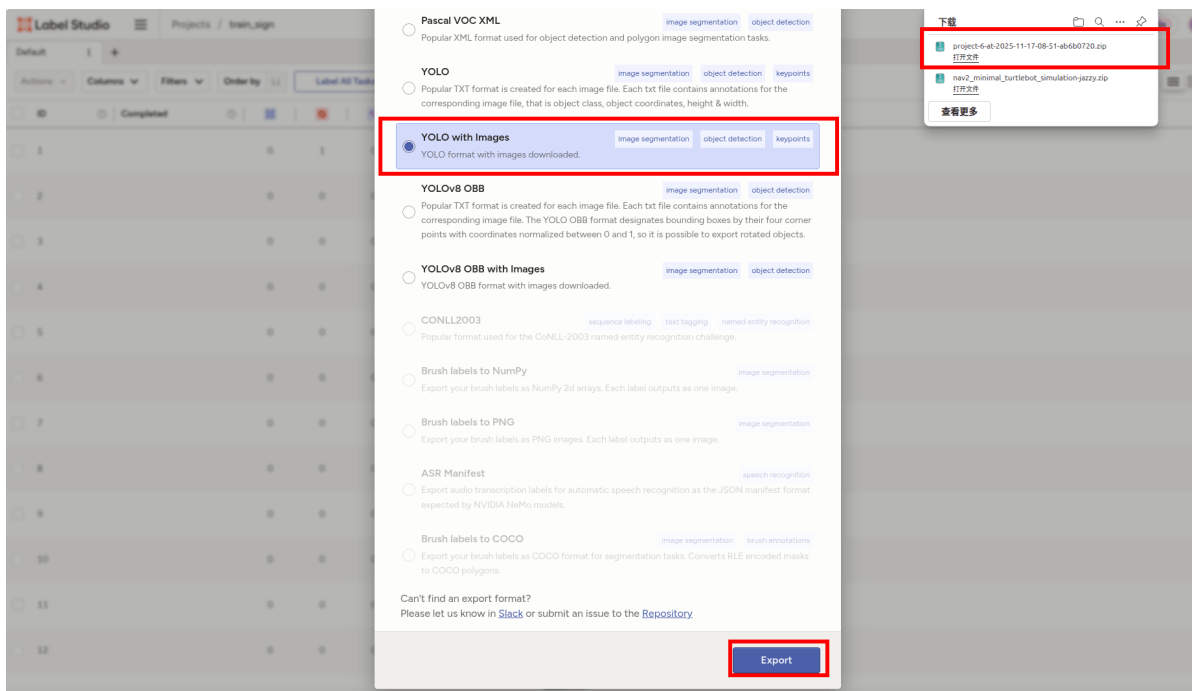
Import

Export

ID	Completed				Annotated by	image	img	
☐ 1	0	1	0	YA			<>	
☐ 2	0	0	0				<>	
☐ 3	0	0	0				<>	
☐ 4	0	0	0				<>	
☐ 5	0	0	0				<>	
☐ 6	0	0	0				<>	
☐ 7	0	0	0				<>	
☐ 8	0	0	0				<>	
☐ 9	0	0	0				<>	
☐ 10	0	0	0				<>	
☐ 11	0	0	0				<>	
☐ 12	0	0	0				<>	
								

Tasks: 62 / 62 Submitted annotations: 0 Predictions: 0

- Select **YOLO with Images** format to export, the browser will automatically download a compressed package



3.9 Organize Dataset Directory Structure

- The directory structure corresponding to the compressed package exported by Label Studio is as follows:

```

.
├── images
├── labels
├── classes.txt
└── notes.json

```

Folder and file descriptions:

- images: Contains original images

- labels: Annotation data corresponding to each image
- classes.txt: Target categories
- notes.json: Target categories and corresponding numbers

- To enable YOLOv11 to recognize the dataset, some adjustments need to be made to the directory structure of data exported from Label Studio
- Split the images and labels exported by Label Studio into train (training set) and val (validation set). The images and labels in train and val need to correspond one-to-one in name.
- Here for the example, we directly copy the images and labels in the train directory into the val directory

```
.
├── classes.txt
├── notes.json
├── train
│   ├── images
│   └── labels
└── val
    ├── images
    └── labels
```

- The following is the directory structure of the tutorial example dataset (the dataset provided in the factory vehicle image and this section tutorial folder is in .zip compressed package format, you need to unzip it first to see the complete directory structure)

A terminal window screenshot showing a directory tree command. The terminal title is 'jetson@yahboom: ~/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset'. The command entered is 'tree -L 2'. The output shows a directory structure with 'classes.txt', 'notes.json', 'train' (containing 'images', 'labels', 'labels.cache'), and 'val' (containing 'images', 'labels', 'labels.cache'). At the bottom, it says '6 directories, 4 files'.

```
jetson@yahboom: ~/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset
jetson@yahboom:~/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset$ tree -L 2
.
├── classes.txt
├── notes.json
├── train
│   ├── images
│   ├── labels
│   └── labels.cache
└── val
    ├── images
    ├── labels
    └── labels.cache
6 directories, 4 files
```

3.10 Dataset Configuration File

- When training YOLOv11 models, you need to specify the dataset path and detection target categories through a yaml format configuration file.
- Traffic sign dataset configuration file path:

```
/home/jetson/yahboomM3_ws/yolo_train/train_sign/sign.yaml
```

Content analysis:

- path: Dataset path
- train: Training set path
- val: Validation set path

- names: Detection target categories (need to be consistent with the sequence categories in notes.json exported from Label Studio)

```
path: /home/jetson/yahboomM3_ws/yolo_train/roadsign_dataset # dataset root dir
train: train/images
val: val/images

# Classes
names:
  0: greenlight
  1: honking
  2: parkA
  3: parkB
  4: redlight
  5: school
  6: sidewalk
  7: sidewalk_ground
  8: speedlimit
  9: stop
  10: straight
  11: turnright
  12: turnright_ground
  13: yellowlight
```

4. Annotating Lane Detection Dataset

The lane detection dataset annotation process is basically the same as the traffic sign dataset annotation process, both use YOLOv11's object detection model for object detection (here it is recommended to use object detection model, not recommended to use segmentation model for lane detection, model inference speed is slower)

4.1 Create Project

- The creation process is consistent with the previous one. Here fill in `train_lane` for Project Name.

Create Project

Project Name Data Import Labeling Setup

Cancel Save

Project Name

train_lane

Description

Optional description of your project

Workspace Enterprise

Select an option

Simplify project management by organizing projects into workspaces. [Learn more](#)

New Storage Connector

You can connect Label Studio Enterprise to Databricks Unity Catalog (UC) Volumes to import files as tasks and export annotations. [Learn more](#)



- Here the lane detection model category has only one, fill in `lane`

Create Project

Project NameData ImportLabeling Setup

CancelSave

Labeling Interface

Browse TemplatesCodeVisual

Configure data

Use image from<set manually>\$image

Add label names

Use new line as a separator to add multiple labels

lane

Add

Labels (1)

lane

Configure settings

Width of region borders1

☐ Allow image zoom (ctrl+wheel)
 ☐ Show controls to zoom in and out
 ☐ Show controls to rotate image

Display labels:bottom

☐ Add filter for long list of labels

UI Preview

- Import the split lane video frame dataset

Import Data

CancelImport

Dataset URL

Add URL

or

Upload More Files

11 files uploaded

Files

upload/7/f5f3e7d5-ebacce16-494.jpg	43.1 KB
upload/7/78f619e3-feca2652-552.jpg	52 KB
upload/7/a484b7d7-fe517684-97.jpg	44.6 KB
upload/7/c9386389-fca94b45-587.jpg	62.07 KB
upload/7/29b1e680-fc450179-10.jpg	47.16 KB
upload/7/8feca5a6-fc31baba-462.jpg	43.27 KB
upload/7/94b68863-fbae8f6b-15.jpg	55 KB
upload/7/47221973-fa645c82-31.jpg	50.09 KB
upload/7/231b5bf0-fa5af7b0-407.jpg	27.32 KB
upload/7/9ce4b882-fa3077a3-507.jpg	39.01 KB
upload/7/f67ef9e-f9bee1d8-569.jpg	59.97 KB

- The annotation method is consistent with the previous one. Here we box and annotate all lane lines

Label Studio

Projects / train_lane / Labeling

SettingsAutoBeta1.9A

#70
8 of 8

InfoHistory

View region details

Select a region to view its properties, metadata and available actions

RegionsRelations

ManualBy Time12

lane

lane

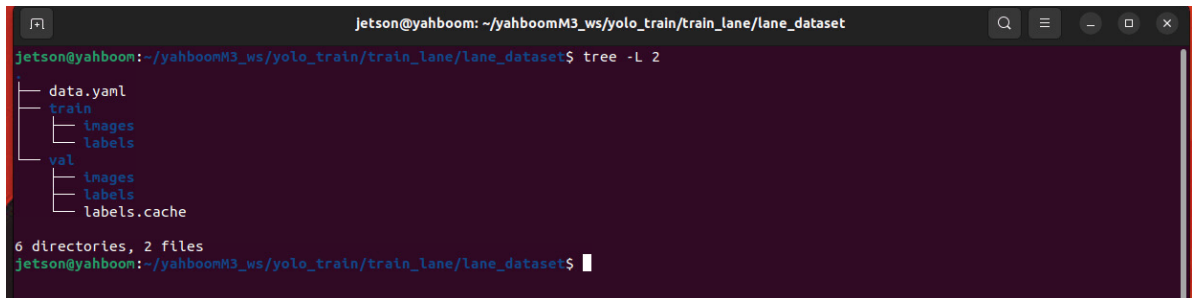
lane

Skip

Submit

4.2 Organize Export Dataset

- After annotation is completed, we export and organize the dataset into a target structure recognizable by YOLO. The method is completely consistent with the previous traffic sign dataset export and organization method.
- Reference target structure is as follows:



```
jetson@yahboom: ~/yahboomM3_ws/yolo_train/train_lane/lane_dataset
jetson@yahboom:~/yahboomM3_ws/yolo_train/train_lane/lane_dataset$ tree -L 2
.
├── data.yaml
├── train
│   ├── images
│   └── labels
└── val
    ├── images
    ├── labels
    └── labels.cache

6 directories, 2 files
jetson@yahboom:~/yahboomM3_ws/yolo_train/train_lane/lane_dataset$
```

4.3 Dataset Configuration File

- Lane detection dataset configuration file path:

```
/home/jetson/yahboomM3_ws/yolo_train/train_lane/lane.yaml
```

- Example configuration file is as follows:

```
path: /home/jetson/yahboomM3_ws/yolo_train/train_lane/lane_dataset # dataset
root dir
train: train/images
val: val/images

# Classes
names:
  0: lane
```